EDUCATION SaaS

Final Reconciled Specification — Schools Module

Multi-tenant school management platform. Consolidates the security review, product decisions, data model, and full role/permission design reconciled across all planning documents and working sessions.

Universities are out of scope. This is the single source of truth — supersedes all prior drafts.

Table of Contents

1. Scope and Product Goals
2. Roles and Permission Model
3. Authentication and Identity
4. Search
5. School Module
6. Attendance System
7. Tests, Marks, Grades and Remarks
8. Database Architecture
9. Security Architecture
10. Data Lifecycle and Archival
11. Backend, API, and Hosting
12. What to Do / What Not to Do
13. Development Phases
14. Deferred / Future
15. Final Implementation Checklist

1. Scope and Product Goals

1.1 Tenant Model

One school = one tenant. Universities are removed entirely from scope — no roles, no feature flags, no beta path. If revisited in future, this is treated as an explicit rewrite, not a migration.

Hierarchy: Tenant → Academic Year → Class → Section. Fixed at two levels below the year. No generic nested org-unit table.

1.2 Roles (7 total)

Layer	Roles
Tenant (School)	School Admin, Principal, Admissions Officer, Teacher (incl. Class Incharge), Student
Platform	Platform Owner, Platform Admin

1.3 Launch Modules (v1)

- Authentication & identity
- Students & teachers
- Classes & sections
- Attendance (student + teacher)
- Leave management
- Tests / marks / grades / remarks
- Timetable
- Reports & dashboard alerts
- Bulk Excel import
- Audit logs
- Notifications (in-app)
- Report card generation

1.4 Client

Browser-based web app built with Next.js. Not a native mobile app. The teacher attendance flow specifically is built as an installable, offline-capable Progressive Web App (PWA) using a service worker — see Section 6.

1.5 Explicitly Out of Scope (v1)

- Universities (any form)
- Fees & defaulter tracking
- Payroll / HR / generic Employee accounts
- Homework/assignment posting
- ID cards & certificates
- Urdu localization
- Native mobile app

- SMS / WhatsApp channels
- AI insights / AI-drafted remarks
- Academic calendar / holiday model
- Biometric / RFID hardware
- In-app chat
- Separate parent portal / parent login
- Multi-factor authentication (MFA)

2. Roles and Permission Model

Permissions are enforced by role + explicit permission grant, checked server-side on every request — never by role name alone, and never trusted from the client.

2.1 School Admin

One School Admin account per school. The only role that can create Principal, Admissions Officer, and all other accounts at that school.

- Students: add / edit / deactivate / bulk import
- Teachers: add / edit / assign to subjects & sections
- Classes, sections, academic year management
- Promotion (whole school, class/section, or individual)
- Attendance oversight (full grid, editable)
- Leave approvals (fallback)
- Marks, grades, remarks (full access)
- Timetable management
- Reports, dashboards, audit logs
- School settings — including toggling what Principal and Admissions Officer can do
- Announcements

Note: If this single account is locked out with no recovery path, the only recourse is Platform Owner via the break-glass support-access flow (Section 15.3).

2.2 Principal

Leadership oversight role. Permissions are configurable per school by that school's own School Admin — not hard-coded. Same role name can mean very different access at two different schools.

School A example	School B example
Add/edit students: ON Manage classes/sections: ON Promote students: ON Approve teacher leave: ON View audit logs: ON	Add/edit students: OFF Manage classes/sections: OFF Promote students: OFF Approve teacher leave: ON View audit logs: ON

Student search is granted by default (read-only), regardless of other toggles — low risk, high utility.

Principal-as-teacher edge case: handled as a runtime check (block self-approval on their own record), not a structural data-model constraint.

2.3 Admissions Officer

Doubles as the platform's only non-teaching-staff role — there is no separate generic "Employee" entity in v1.

- Add student (individual)
- Bulk import students
- Validate imported records
- Assign class/section
- Generate username
- Account activation / reset workflow

- View admissions-scoped student list

2.4 Teacher

Access is fixed by the system, scoped to assigned classes/subjects only — not admin-configurable.

- Mark attendance for assigned sections (batch submit)
- Enter marks for assigned assessments (batch submit)
- Add remarks
- Apply for own leave
- View assigned timetable
- Search students within assigned sections only
- Mark own (teacher) attendance — self check-in
- Direct-edit own submitted attendance within 3-day rolling window
- Direct-edit own submitted marks within 7-day per-assessment window

2.5 Class Incharge

Not a separate role — a designation. section.class_teacher_id points to one Teacher per section, set by School Admin. Distinct from subject-level teacher_assignments.

- Full read/write access to their own section's attendance grid
- Moderate remarks for their own section
- View their own section's timetable
- Trigger roll-number renumbering for their section (with mandatory confirmation)
- Leave approval fallback for their section's students if no incharge assigned / request ages out

Note: Eligibility to become Class Incharge is not hard-restricted to teachers already assigned to that section, but the picker UI defaults to showing them first.

2.6 Student

Access is fixed by the system, scoped to own records only.

- View own attendance, marks, grades, remarks, timetable
- Apply for leave

Note: No separate parent login. Parents use the student's own account. Guardian name/phone are stored as reference data on the student record (Section 5.5), not as a login identity.

2.7 Platform Owner

Owns the SaaS business. Highest-risk account class.

- Create / suspend / manage tenants
- Manage plans & subscriptions
- Manage Platform Admin accounts and their permission scope
- Request time-boxed, audited support access into a tenant (break-glass — see 15.3)
- View platform-wide audit logs, system health
- Search tenants (schools) by name/metadata — does not expose a tenant's internal student/teacher data

2.8 Platform Admin

Restricted operational seat. No tenant data access by default, no tenant creation/deletion, no billing.

- Routine operational tasks as explicitly granted by Platform Owner
- System health monitoring
- Support ticket queue

3. Authentication and Identity

3.1 Two Parallel Login Paths, One Endpoint

A single login form. The server tries a username lookup first, falls back to email lookup — both resolve to the same users table and same session mechanism afterward.

User type	Credential
Student, Teacher	Username + password (custom system, no email required)
School Admin, Principal, Admissions Officer	Email + password via Supabase Auth
Platform Owner, Platform Admin	Email + password via Supabase Auth

3.2 Username Scheme (Students)

Format: prefix-class/section-name-rollnumber, e.g. bss-9a-ahmad-14

- Prefix is chosen by each school at tenant setup and must be unique across the whole platform (validated at creation — format restricted to letters/numbers, no spaces).
- Regenerated automatically at promotion (class/section segment changes) and at manual roll-number renumbering (roll-number segment changes).
- No username history table kept. The moment a username changes, the old one stops working immediately — no grace period.
- No automatic notification on change (deliberately dropped). Recovery path if a student is locked out: School Admin/Principal search by name to find the current username.
- Case-insensitive and separator-insensitive at login: BSS-8A-Ahmad-14, bss-8a-ahmad-14, and bss8aahmad14 all resolve to the same account.
- Implemented via a stored, indexed username_normalized column (lowercase, hyphens stripped) used for the actual lookup — keeps login fast at any scale. Uniqueness constraint lives on this normalized column.

3.3 Roll Numbers

- Assigned sequentially as students join a class — no automatic gap-filling.
- Sticky by default: a student leaving, failing, or repeating does NOT trigger renumbering for anyone else. Gaps persist.
- At promotion: roll number carries over unchanged into the new class/section (only the class/section part of the username changes).
- Manual "Renumber roll numbers" action — available to Class Incharge for their own section, and to School Admin for any section — resequences 1, 2, 3... Requires mandatory confirmation showing how many usernames will change. No notification sent. Fully audit-logged (before/after mapping for every affected student).

3.4 Sessions

- Session rows already exist in the Identity table group (see Section 8).
- Access token: 1 hour, auto-refreshed silently by the client SDK.
- Refresh token ("stay logged in") expiry: 60–90 days of inactivity — deliberately raised from Supabase's 30-day default, so normal periodic use never triggers a re-login prompt.

- Invalidation: on password change, role change, or account deactivation, delete that user's active session row(s) directly (or revoke via Supabase's admin API for email-auth accounts). Forces re-login on next request. No session-version counter — deliberately simplified.
- Explicit logout revokes the refresh token server-side, not just a client-side cookie clear.

3.5 Password Reset

Relies on Supabase Auth's built-in implementation for email-based accounts (School Admin, Principal, Admissions Officer, Platform Owner/Admin): single-use token, short expiry, no account-existence disclosure on the response.

Students and Teachers have no email on file. Reset is admin-only: School Admin resets the credential and relays it directly (reusing the existing forced-password-change-on-first-login pattern already used for onboarding). No self-service path for these roles in v1.

3.6 Multi-Factor Authentication

Not implemented in v1. Considered and explicitly dropped for simplicity across all roles, including Platform Owner.

4. Search

Same underlying pattern (auth → tenant resolution → role check → scoped query) applies to every search endpoint, with results filtered further by role.

Who searches	Searches for	Scope
Teacher	Students	Only sections they are assigned to (via teacher_assignments) — never school-wide
School Admin	Students, Teachers	Whole school. Class/section filter is optional, not required
Principal	Students	Whole school, read-only result. Optional class/section filter. Granted by default, not toggle-gated
Platform Owner	Schools (tenants)	Whole platform — searches tenant name/metadata only, never a tenant's internal student/teacher data

- Implemented with Postgres pg_trgm for fuzzy/typo-tolerant name matching.
- All search endpoints are paginated (20 results at a time).
- Same-name students are never ambiguous in results: every row shows class/section and username alongside the name, so two students named Ahmad Khan are always distinguishable.
- Same-name collisions no longer need special-case handling in usernames — the roll-number segment in the username format already guarantees uniqueness.

5. School Module

5.1 Student Management

- Add individually or via bulk Excel import
- Edit / deactivate (soft delete, 30-day restore window)
- Assign to class/section
- Guardian name and phone stored on the student record (Section 5.5)
- Search by name, filterable by class/section/status

5.2 Teacher Management

- Add / edit teachers
- Subject-level assignment: teacher_assignments links teacher + section + subject (many-to-many). This is what powers all attendance/marks/search scoping elsewhere in the system.
- Class Incharge designation is separate — a single field on the section, not derived from subject assignment.

5.3 Classes, Sections, Academic Year

- Academic year is a first-class concept — classes and sections exist within a year.
- Promotion: bulk, per-class/section, or individual. Roll number carries over unchanged; only class/section changes.
- Section transfers / mid-year changes: supported as an explicit School Admin action.

5.4 Promotion — Full Behavior

Promotion is the single most consequential action in the system — it triggers username regeneration and the end-of-year data archival described in Section 10.

- 1. School Admin runs promotion for a class/section (or the whole school).
- 2. For each promoted student: username regenerates (class/section segment only; roll number unchanged).
- 3. For each promoted student: a full Report Card PDF for the completed year is generated and uploaded to Cloudflare R2.
- 4. Once upload is confirmed, that student's attendance and marks records for the completed year are deleted from Postgres.
- 5. No blocking check against attendance's 3-day correction window — if promotion runs before the window closes on the last few days, those days simply don't get a chance to be corrected. Accepted trade-off.

Note: Mid-year withdrawal (a student leaving without being promoted) does NOT trigger report-card generation. The student's data simply follows the standard retention/anonymization path with no archival step.

5.5 Guardian Details

Fields on the students table: guardian_name, guardian_phone. Kept as two plain columns, not a separate guardians entity — deliberately simple.

- Visible to: School Admin, Admissions Officer, Principal (read), Teacher (their assigned students only, for contact purposes).
- Editable by: School Admin only.
- Manual-reference data only — no automatic SMS/call is triggered by this field (that channel is out of scope).

- Searchable by School Admin/Admissions Officer (e.g. a parent calls in, look up which student the number belongs to).
- Included in anonymization when a student's record is anonymized on withdrawal.
- Excluded from general exported reports/CSVs by default — admin-only visibility for exports.

6. Attendance System

6.1 Student Attendance — Design Philosophy

Not a running counter. Every submission is a row: (student_id, section_id, subject_id, date, status). Attendance percentage is computed by counting rows at read time, never incrementally maintained — this is what makes corrections, duplicate-detection, and per-day lookups all work correctly and stay in sync automatically.

```
SELECT COUNT(*) FILTER (WHERE status='present') AS present,        COUNT(*) AS total
FROM attendance_records WHERE student_id = X AND date BETWEEN start AND end
```

6.2 Offline-First Design (Teachers Only)

The backend is the security boundary — every write is server-validated. Without offline support, a teacher with no signal cannot submit attendance at all. This is solved for teacher-facing actions only (attendance, marks entry), not for admin actions.

- PWA app shell (next-pwa / Workbox) caches the app shell and the teacher's assigned roster on login.
- Local queue: IndexedDB (e.g. Dexie.js). Taps only update local React state — zero API calls per tap.
- One batch submission per class per day, not per student.
- Submission queues locally as a single item with status: pending_sync, a client-generated UUID (used as the idempotency key), and a local timestamp.
- Background Sync (or poll-on-reconnect) auto-syncs the queued batch when connectivity returns — this is the one thing that runs in the background via the service worker, even after the app is closed.
- Every synced record is fully re-validated server-side on arrival — auth, tenant, section assignment, correction-window check. An on-device "already validated" flag is never trusted.
- Conflict handling: last-write-wins by server-received time; both attempts logged to audit_logs.
- Late syncs are marked synced_late: true with both the original and server-received timestamps, shown in the UI.

6.3 Batch Submission

```
POST /v1/attendance {   "classId": "...", "sectionId": "...", "subjectId": "...",
"date": "2026-09-03", "clientUuid": "...",   "records": [{ "studentId": "...",
"status": "present" }, ...] }
```

- Server validates the whole batch in one transaction: one tenant/role check, one write, one audit-log entry.
- Warns before submit if any student is unmarked ("3 students not marked — submit anyway?").
- Submit button is disabled while a submission for the same classId-sectionId-subjectId-date key is still pending sync — this, not a server response, is what actually prevents a double-submit under network lag.
- Server-side backstop: a unique constraint on (tenant_id, class_id, section_id, subject_id, date) rejects a true duplicate cleanly even if two devices somehow both queue the same submission.

6.4 Attendance Correction

- The teacher who submitted a record can directly edit it themselves — no request, no approval step, no Principal involvement.
- Window: 3 days, rolling, per individual date. Each day's record has its own independently expiring window — Sept 1st locks after Sept 4th, Sept 2nd locks after Sept 5th, and so on.
- After the window closes: only School Admin can edit that record.
- Every direct edit — within or past the window — writes to audit_logs with who, when, and before/after values.

6.5 Teacher (Staff) Attendance

Self check-in. A teacher marks their own attendance (present/absent/late) — typically on login or via a one-tap action on their Today view.

- Record includes a real timestamp (checked_in_at) — unlike student attendance, this is a genuine point-in-time event, not a daily count.
- Same offline queue mechanism applies if marking from a low-connectivity area.
- Known trade-off, accepted: nothing prevents a teacher from marking themselves present without physically being there. No verification hardware (geofencing, photo check-in) is built for this in v1.
- School Admin / Principal can see a daily view of who checked in and directly correct it — oversight without added verification infrastructure.
- No separately defined correction window for teacher attendance — School Admin can amend it directly at any time.

6.6 Attendance Mode

Per-period or once-daily (first period) — configurable by School Admin per school. Default: once-daily.

7. Tests, Marks, Grades and Remarks

7.1 Assessment Types and Mark Entry

Teacher enters marks locally (draft), submits (submitted, locked to the teacher), and — where the publication toggle is enabled — School Admin or Principal publishes (published) to students/parents. Publication default: School Admin; toggleable to Principal per school.

7.2 Grading

Grading scale is versioned per academic year — adjusting grade boundaries never silently rewrites a prior year's already-issued transcripts.

7.3 Marks Correction

- Identical mechanism to attendance correction, kept as a fully separate implementation (different table, different granularity) — not merged into one shared "corrections" system.
- The submitting teacher can directly edit marks for any test/assessment, no approval required.
- Window: 7 days per test/assessment, from that assessment's submission — independent per assessment, not a fixed batch date.
- After 7 days: School Admin only.
- Every direct edit writes to audit_logs with who, when, and before/after values.

7.4 Remarks

Moderated by Class Incharge for their own section; School Admin can moderate remarks school-wide.

8. Database Architecture

Structural multi-tenancy, not regional: one shared Postgres database (Supabase), every tenant-owned table carries a tenant_id column, and Row-Level Security (mandatory) enforces that a query can only return rows matching the authenticated user's tenant. All schools share the same database and region.

Group	Tables
Tenancy	tenants, tenant_settings (attendance_mode, at-risk threshold, permission toggles)
Identity	users, roles, permissions, role_permissions, sessions
People	students, teachers — no generic employees table (Admissions Officer covers non-teaching staff)
Structure	academic_years, classes, sections (incl. class_teacher_id for Class Incharge), subjects, teacher_assignments
Placement	current student placement (section_id on the student record)
Attendance & leave	attendance_records (current year only — see Section 10), attendance_rules, leave_requests, leave_types, leave_approvals
Assessment	assessments, marks (current year only — see Section 10), grading_schemes (versioned by year), remarks
Scheduling & comms	timetable_slots, timetable_entries, announcements, notifications
Ops & security	import_jobs, import_errors, audit_logs (archived monthly to R2 — see Section 10), support_grants
SaaS	plans, subscriptions, usage_counters

Every tenant-scoped table carries tenant_id with an RLS policy, plus deleted_at for the 30-day soft-delete restore window.

8.1 Username Storage

```
students   username            -- BSS-8a-Ahmad-14   (canonical/display)
username_normalized   -- bss8aahmad14      (lowercase, hyphens stripped, indexed,
unique)
```

8.2 Deliberately Absent

- No university tables of any kind (faculties, departments, programs, courses, semesters, GPA/CGPA).
- No username_history table.
- No session_version column.
- No Redis-backed rate-limit store — handled via two columns on users instead (Section 9.4).
- No generic employees table.
- No permanent per-year attendance/marks storage — see Section 10.

9. Security Architecture

9.1 Request Pipeline

```
Request → HTTPS → Rate limit → Authentication → Session validity → Tenant resolution → Role/permission authorization → Input validation → Business rule checks → Tenant-scoped DB query → Audit log (if required) → Response
```

9.2 Authorization & Tenancy

- Never trust tenant_id, role, or resource IDs (student_id/teacher_id) supplied by the browser.
- Every query scoped by the authenticated tenant.
- PostgreSQL Row-Level Security is MANDATORY on every tenant-scoped table — a second, database-level enforcement layer independent of application code, not "where appropriate."
- Dedicated authorization test suite: for each role, attempt (a) access to a same-tenant peer's record and (b) access to a cross-tenant record. Timetable and report endpoints included explicitly — they aggregate across sections and are the likeliest accidental leak point.

9.3 Authentication Security

- Argon2id (or equivalent modern) password hashing — never plaintext.
- Secure, HTTP-only cookies; Secure + SameSite settings.
- Force password change for any temporary/admin-issued credential on first login.
- No MFA in v1 (see Section 3.6).
- Sessions invalidated (rows deleted / Supabase session revoked) on password change, role change, or deactivation.

9.4 Rate Limiting & Brute-Force Protection

No Redis/Upstash store — deliberately kept out of the stack. Handled directly in Postgres:

- failed_login_attempts and locked_until columns on users. 5 failed attempts → 15-minute lock.
- Broad API abuse protection relies on Vercel's built-in edge-level firewall/bot protection — no per-endpoint application store needed.

9.5 Input & Upload Security

- Validate all request bodies server-side; enforce numeric ranges for marks, valid dates for leave/attendance.
- Validate uploaded file type by content-sniffing (not extension or client-declared MIME type), plus size cap.
- Sanitize filenames; use generated object keys for R2 storage, never user-supplied names.
- Never execute or store uploaded files in an executable path.
- Bulk Excel import: strip/quote cells starting with =, +, -, @, tab, CR (formula injection) on both import and export. Cap decompressed size and row/cell count before full parse (a .xlsx is a zip — zip-bomb risk). Disable external entity resolution in the spreadsheet parser (XXE). Parse in a separate worker with memory/time limits. Idempotency key per import job so a network retry doesn't create duplicate students.

9.6 Platform-Owner Support Access (Break-Glass)

Even Platform Owner does not get silent access to tenant data:

- 1. A support request comes in from a school.

- 2. Platform Owner explicitly requests time-boxed access to that tenant, with a stated reason and an expiry (e.g. 2 hours).
- 3. The system logs who, which tenant, why, start/end time.
- 4. Access auto-expires — does not linger indefinitely.
- 5. The tenant's own audit log shows the access occurred.

Platform Admin gets zero tenant-data access by default; any support employee needing it goes through the identical flow with Platform Owner as approver.

9.7 Audit Logs

Insert-only DB role — the application's own DB credential has no UPDATE/DELETE grant on audit_logs, so a compromised app credential cannot tamper with its own trail.

Logged events: Login/logout, student creation/update/deactivation, attendance edits, marks/grade changes, leave approvals/rejections, promotion actions, roll-number renumbering, permission changes, tenant suspension/reactivation, support access, photo uploads, guardian-detail edits.

Never logged: plaintext passwords, session secrets, R2/DB credentials, or any other secret.

Retention: archived to Cloudflare R2 monthly, per tenant, then deleted from Postgres. See Section 10.3.

10. Data Lifecycle and Archival

Deliberately designed to keep Postgres storage flat over time, appropriate for Supabase's free-tier 500MB limit during the prototype phase, without losing any data that's actually needed — everything closed out simply moves to Cloudflare R2 rather than being deleted outright.

10.1 Why Row-Per-Day, Not a Running Counter

At full scale (1,000 students, once-daily attendance, ~200 school days/year) attendance is roughly 200,000 rows and ~30MB per year — negligible against a 500MB budget. A running counter would save perhaps 100 bytes/record while breaking corrections, duplicate-detection, and per-day lookup entirely. Row-per-day is correct regardless of storage concerns.

10.2 Attendance & Marks — Archived at Promotion

- Full academic year stays live and fully queryable in Postgres throughout the year — corrections, at-risk alerts, and dashboards all work off real data.
- At promotion (Section 5.4), for each student: a full Report Card PDF (marks + grades + attendance % + remarks) is generated and uploaded to R2: reports/{tenant}/{year}/{student}-report-card.pdf.
- Once the upload is confirmed, that student's attendance and marks rows for the completed year are deleted from Postgres.
- Closed years exist only as the report card PDF from that point forward — no per-date lookup, no re-generation in a different format, no summary table kept in Postgres.
- Accepted trade-off: if a report-generation bug is later discovered, there is no raw data left to regenerate a corrected report from for a closed year.

10.3 Audit Logs — Archived Monthly

- audit_logs stays live in Postgres for the current month only.
- Monthly scheduled job exports the previous month's rows per tenant to a compressed file in R2: audit-archives/{tenant}/2026-08.json.gz.
- Once upload is confirmed, those rows are deleted from Postgres.
- Old audit trails (e.g. a support-access dispute from months ago) are retrieved by pulling and searching the R2 file — slower, but not lost. This preserves the accountability guarantee behind the break-glass flow (Section 9.6) even though the data isn't a live query after a month.
- Archived per-tenant-per-month (one file covering all activity for that school that month), not per-user — matches how audit queries are actually used ("what happened in this school in this window").

10.4 Explicitly Not Archived

- Mid-year student withdrawal — no report card generated, no special archival step. The record simply follows the standard retention/anonymization path.
- Backups — no backup plan exists yet for the prototype phase. Deliberately deferred.

11. Backend, API, and Hosting

11.1 Backend Framework — Next.js Route Handlers (Serverless)

Locked. Not NestJS, not a separate Fastify service.

- One deployment target (Vercel) — no separate backend service to host or scale independently.
- Prisma + Supabase Postgres integrate natively, no adapter layer.
- RBAC/tenant checks live in shared middleware used by every route handler.
- API is versioned (/v1/...) — if a specific workload later outgrows serverless execution limits, only that route extracts to a standalone service; the frontend never notices, since it only ever talks to /v1/...

11.2 What "Serverless" Means (Practical Implications)

- No always-on server — each route is a function, invoked on request, then shut down. Pay per invocation, not per hour.
- No persistent memory between requests — this is why rate limiting uses Postgres columns rather than in-process counters (Section 9.4).
- Execution time limits (10s free tier) — this is why long-running tasks (Section 11.4) must not run inline.
- Cold starts after idle time — not a practical issue at this traffic pattern.

11.3 Scheduled Jobs — Vercel Cron

Config-driven, not a running process. vercel.json defines a schedule; Vercel calls the route at that time.

```
// vercel.json { "crons": [{ "path": "/api/cron/at-risk-attendance", "schedule": "0 6 * * *" }] }
```

- Used for: daily at-risk-attendance checks, the monthly attendance/marks and audit-log archival jobs (Section 10), expired password-reset-token cleanup.
- Each cron route is secured — rejects requests unless they include the secret header only Vercel's cron system knows.
- Free tier: daily granularity only, small number of scheduled jobs — sufficient for this system's needs.

11.4 Long-Running Tasks — Deferred Detail

Bulk Excel import and batch report-card PDF generation can exceed serverless execution limits. General direction (revisit in detail before Phase 4 build): request returns immediately with a job record ("processing"); a queue (e.g. Upstash QStash) or Vercel Background Function performs the actual work outside the request/response cycle; frontend polls or is notified when the job completes.

11.5 API Design Rules

- Versioned contract (/v1/...).
- Consistent status codes and error shapes.
- Validate at the API boundary.
- Never return more data than the current user's role needs.
- Pagination for all list/collection endpoints.
- Explicit filters rather than downloading entire tenant datasets.
- Authorization test for every protected endpoint.

11.6 Hosting — Prototype Target

Layer	Provider
Frontend + serverless API	Vercel (Next.js)
Database	Supabase PostgreSQL
File storage	Cloudflare R2 (photos, report cards, archived audit logs)
DNS / TLS / CDN	Cloudflare
Source control	GitHub
Error monitoring	Sentry

Note: Free tiers can change and should be rechecked before committing to production usage. Not treated as production-compliant for official educational records until backups, monitoring, and final security review are in place.

11.7 Development Workflow

Cloud-only development environment (Codespaces/devcontainer), switchable to local via a single flag — same devcontainer.json works in both contexts, no rewrite needed to switch.

- Storage abstraction: a StorageService interface with Local and R2 implementations, so moving from local dev storage to R2 doesn't require rewriting student/teacher modules.
- CI/CD: GitHub Actions triggers Vercel preview deploys per pull request; merging to main auto-deploys.
- Every protected endpoint requires an authorization test before being considered done (Section 9.2).

12. What to Do / What Not to Do

12.1 Do

- Keep the browser untrusted; enforce every rule server-side.
- Scope every tenant-owned query by authenticated tenant.
- Use PostgreSQL for relational academic data; R2 for images/files, never database blobs.
- Use pagination and indexed queries throughout.
- Make Principal's permissions configurable, not hard-coded.
- Audit every sensitive mutation.
- Preview/crop photos before final save.
- Treat academic year / class / section as first-class concepts.
- Allow bulk imports with validation and preview before commit.
- Keep reports and heavy jobs separable so they can become background tasks.

12.2 Do Not

- Do not expose database, R2, or server secrets to the browser.
- Do not trust tenant_id, role, or user IDs supplied by the frontend.
- Do not store passwords as plaintext.
- Do not store original high-resolution profile photos unnecessarily.
- Do not attach teacher attendance records to the student data model — kept fully separate.
- Do not store timetable rows separately per student — store per class/section.
- Do not hard-code one grading system for every school.
- Do not let a hidden UI button act as a security control.
- Do not silently overwrite published attendance or marks — corrections go through the direct-edit-within-window mechanism, always audit-logged.
- Do not promise free-tier hosting can support unlimited production schools.
- Do not build a generic employees table for v1 — Admissions Officer already covers the one non-teaching role in scope.

13. Development Phases

Phase 1 — Foundation

Project structure, database schema, RLS policies, authentication (both login paths), sessions, tenant resolution, roles/permissions, Platform Owner, organization creation.

> *Note: Acceptance: a user can log in; session persists securely; user sees only the correct dashboard; cross-tenant access attempts are rejected and logged.*

Phase 2 — School Administration

Students, teachers, teacher_assignments, classes, sections, subjects, academic years, promotion (incl. username regeneration + report-card archival), profile photos, Class Incharge designation, guardian fields.

Phase 3 — Academic Operations

Attendance (student batch-submit + offline PWA, teacher self check-in), leaves, assessments, marks, grades, remarks, correction windows (3-day rolling / 7-day per-assessment), timetable + conflict validation.

Phase 4 — Management Operations

Reports, dashboard alerts, announcements, bulk Excel import (with Section 9.5 hardening), search (all three role scopes), audit logging, exports.

Phase 5 — Production Hardening

Rate-limit columns, security headers (CSP/HSTS/X-Frame-Options), IDOR authorization test suite, break-glass support access, roll-number renumbering, monthly archival jobs (attendance/marks at promotion, audit logs monthly), Sentry, CI dependency scanning.

14. Deferred / Future (Not Blocking Phase 1)

Item	Status
Universities (any form)	Fully out of scope — explicit rewrite if ever revisited
Fees & defaulter tracking	Future module
Payroll / HR / generic Employee accounts	Future — not needed while Admissions Officer covers non-teaching staff
Backups	No plan yet — deliberately deferred
Privacy policy & Terms of Service	To be written later — not blocking
Long-running task queue (bulk import / PDF gen)	Direction set (Section 11.4), detailed design deferred to pre-Phase-4
Notification channels beyond in-app	Pluggable interface planned; email/SMS deferred
MFA	Considered and dropped
Native mobile app	Not planned — PWA covers the one workflow (attendance) that needed offline support
Homework/assignments, ID cards, certificates, chat, Urdu localization	Deferred

15. Final Implementation Checklist

Architecture

- ☐ Multi-tenant boundary exists and is enforced by RLS
- ☐ Backend (Next.js route handlers) is the sole security boundary
- ☐ Database is PostgreSQL (Supabase)
- ☐ Storage is abstracted (Local/R2) via StorageService
- ☐ API is versioned (/v1/...)

Authentication

- ☐ Role is never selected manually at login — resolved server-side
- ☐ Student/teacher usernames follow the class-embedded, roll-number-suffixed convention
- ☐ Username lookup is case-insensitive and separator-insensitive (normalized column)
- ☐ Admin-tier accounts use Supabase Auth (email)
- ☐ Sessions persist 60–90 days inactivity; revoked on password/role change or deactivation
- ☐ No MFA implemented (by design)

School Module

- ☐ Students, Teachers, Classes, Sections, Subjects, Academic Years
- ☐ Promotion — regenerates usernames, archives report card, deletes year's raw records
- ☐ Roll-number renumbering — confirmation required, no notification, audit-logged
- ☐ Attendance — batch submit, offline PWA queue for teachers, 3-day rolling correction window
- ☐ Teacher self check-in attendance
- ☐ Marks/Grades/Remarks — 7-day per-assessment correction window
- ☐ Timetable with server-side conflict validation
- ☐ Reports, dashboard alerts, announcements
- ☐ Bulk import with formula-injection and zip-bomb protection
- ☐ Audit logs — insert-only role, archived monthly to R2
- ☐ Guardian name/phone on student record
- ☐ Search — role-scoped (own sections / whole school / whole platform)

Security

- ☐ Tenant scoping on every query
- ☐ RLS mandatory on every tenant-scoped table
- ☐ Permission checks server-side, never trusting client-supplied role/tenant/IDs
- ☐ Input validation at the API boundary
- ☐ Login rate limiting via Postgres columns (failed_login_attempts, locked_until)
- ☐ Secure, HTTP-only, SameSite cookies
- ☐ Argon2id password hashing
- ☐ Upload validation by content-sniffing
- ☐ Break-glass support access — time-boxed, audited, tenant-visible
- ☐ Dependency scanning (Dependabot/Snyk) in CI

Prototype Deployment

- ☐ Vercel configured (frontend + serverless API)
- ☐ Supabase PostgreSQL configured with RLS policies
- ☐ Cloudflare R2 configured (photos, report cards, audit archives)
- ☐ Environment secrets stored server-side only
- ☐ GitHub repository protected, CI checks required before merge
- ☐ Sentry error monitoring wired up
- ☐ Backups — explicitly deferred, revisit before real school data goes in production